

get

-
-

get std::get

```
template <typename... Ts>
struct WeirdTuple : private std::tuple<Ts...> {
    using std::tuple<Ts...>::tuple;
};

template <std::size_t I, typename... Ts>
auto get(WeirdTuple<Ts...>& t) { return I + 10; }

int main() {
    WeirdTuple<int> wt(1);

    get<0>(wt); // changing this 0 to anything else will break
               // theoretically, that should still work -
               // we're obviously using the WeirdTuple overload of get<>(),
               // but the compiler fails when trying to compile every possible overload!
}
```

```
get<1>(wt); // error: static assertion failed: tuple index is in range
           // in instantiation of 'std::tuple_element<1, std::tuple<int>>'
           // required by substitution of
           // constexpr std::tuple_element_t<I, std::tuple<Ts...>>&
           // std::get(std::tuple<Ts...>&)
           // [with I = 1; Ts = {int}]
```

std::get

WeirdTuple

std::get

std::get

•
•
•

get

```
WeirdTuple<int> wt(1);  
get<0>(wt); // ok, returns 10  
get<1>(wt); // ok too, returns 11
```

```
template <std::size_t I, class Tuple>  
concept has_get = requires(Tuple& t) { get<I>(t); };  
  
static_assert(has_get<0, std::tuple<int>> == true, "elem 0: int");  
static_assert(has_get<1, std::tuple<int>> == false, "out-of-bounds");
```

get

std::get

```
namespace ud {  
    struct foo { /*...*/ };  
  
    template <std::size_t I>  
    void get(std::any thing) { /*gotten*/ }  
}  
  
std::tuple<ud::foo> t;  
get<0>(t); // ok  
get<1>(t); // error: call to deleted function 'get'  
           // note: declared here  
           //     std::get(std::tuple<Ts...>&) = delete;  
           //     [with I = 1; Ts = {ud::foo}]
```

<utility>

```
// 23.4.4, tuple-like access to pair  
template<class T> class tuple_size;  
template<size_t I, class T> class tuple_element;  
  
template<class T1, class T2> struct tuple_size<pair<T1, T2>>;  
template<size_t I, class T1, class T2> struct tuple_element<I, pair<T1,  
  
template<size_t I, class T1, class T2>
```

```
constexpr tuple_element_t<I, pair<T1, T2>> see below&& get(pair<T1, T2>&
template<size_t I, class T1, class T2>
constexpr tuple_element_t<I, pair<T1, T2>> see below&& get(pair<T1, T2>&
template<size_t I, class T1, class T2>
constexpr const tuple_element_t<I, pair<T1, T2>> see below&& get(const p
template<size_t I, class T1, class T2>
constexpr const tuple_element_t<I, pair<T1, T2>> see below&& get(const
template<class T1, class T2>
constexpr T1& get(pair<T1, T2>& p) noexcept;
template<class T1, class T2>
constexpr const T1& get(const pair<T1, T2>& p) noexcept;
template<class T1, class T2>
constexpr T1&& get(pair<T1, T2>&& p) noexcept;
template<class T1, class T2>
constexpr const T1&& get(const pair<T1, T2>&& p) noexcept;
template<class T2, class T1>
constexpr T2& get(pair<T1, T2>& p) noexcept;
template<class T2, class T1>
constexpr const T2& get(const pair<T1, T2>& p) noexcept;
template<class T2, class T1>
constexpr T2&& get(pair<T1, T2>&& p) noexcept;
template<class T2, class T1>
constexpr const T2&& get(const pair<T1, T2>&& p) noexcept
template<class T, class T1, class T2>
constexpr T& get(pair<T1, T2>& p) noexcept;
template<class T, class T1, class T2>
constexpr const T& get(const pair<T1, T2>& p) noexcept;
template<class T, class T1, class T2>
constexpr T&& get(pair<T1, T2>&& p) noexcept;
template<class T, class T1, class T2>
constexpr const T&& get(const pair<T1, T2>&& p) noexcept
```

pair

```
template<class T1, class T2>
    struct tuple_size<pair<T1, T2>> : integral_constant<size_t, 2> { };

tuple_element<I, pair<T1, T2>>::type
```

```
I < 2                                I
```



```
T1  I == 0                            T2
```

```
template<size_t I, class T1, class T2>
constexpr tuple_element_t<I, pair<T1, T2>>V& get(pair<T1, T2> p) noexcept
template<size_t I, class T1, class T2>
constexpr const tuple_element_t<I, pair<T1, T2>>V& get(const pair<T1, T2> p) noexcept
template<size_t I, class T1, class T2>
constexpr tuple_element_t<I, pair<T1, T2>>V& get(pair<T1, T2>&& p) noexcept
template<size_t I, class T1, class T2>
constexpr const tuple_element_t<I, pair<T1, T2>>V& get(const pair<T1, T2> p) noexcept
```

```

I == 0                                p.first  I == 1
p.second

```

I < 2	V tuple element t<I, pair<T1, T2>>
	V

```
template<class T1, class T2>
constexpr T1& get(pair<T1, T2>& p) noexcept;

template<class T1, class T2>
constexpr const T1& get(const pair<T1, T2>& p) noexcept;

template<class T1, class T2>
constexpr T1&& get(pair<T1, T2>&& p) noexcept;
```

```
template<class T1, class T2>
constexpr const T1&& get(const pair<T1, T2>&& p) noexcept;
```

T1 T2

p.first

```
template<class T2, class T1>
constexpr T2& get(pair<T1, T2>& p) noexcept;
template<class T2, class T1>
constexpr const T2& get(const pair<T1, T2>& p) noexcept;
template<class T2, class T1>
constexpr T2&& get(pair<T1, T2>&& p) noexcept;
template<class T2, class T1>
constexpr const T2&& get(const pair<T1, T2>&& p) noexcept;
```

T1 T2

p.second

```
template<class T, class T1, class T2>
constexpr T& get(pair<T1, T2>& p) noexcept;
template<class T, class T1, class T2>
constexpr const T& get(const pair<T1, T2>& p) noexcept;
template<class T, class T1, class T2>
constexpr T&& get(pair<T1, T2>&& p) noexcept;
template<class T, class T1, class T2>
constexpr const T&& get(const pair<T1, T2>&& p) noexcept;
```

```
is same v<T, T1> true p.first
is same v<T, T2> true p.second
```

T1 T2

T T1 T2

<tuple>

```
// 23.5.3.7, element access
template<size_t I, class... Types>
constexpr tuple_element_t<I, tuple<Types...>> see below& get(tuple<Types...>& t) noexcept;
template<size_t I, class... Types>
constexpr tuple_element_t<I, tuple<Types...>> see below&& get(tuple<Types...>&& t) noexcept;
template<size_t I, class... Types>
constexpr const tuple_element_t<I, tuple<Types...>> see below& get(const tuple<Types...>& t) noexcept;
template<size_t I, class... Types>
constexpr const tuple_element_t<I, tuple<Types...>> see below&& get(const tuple<Types...>&& t) noexcept;
template<class T, class... Types>
constexpr T& get(tuple<Types...>& t) noexcept;
template<class T, class... Types>
constexpr T&& get(tuple<Types...>&& t) noexcept;
template<class T, class... Types>
constexpr const T& get(const tuple<Types...>& t) noexcept;
template<class T, class... Types>
constexpr const T&& get(const tuple<Types...>&& t) noexcept;
```

```
template<size_t I, class... Types>
constexpr tuple_element_t<I, tuple<Types...>> V&
get(tuple<Types...>& t) noexcept;
template<size_t I, class... Types>
constexpr tuple_element_t<I, tuple<Types...>> V&&
```

```

get(tuple<Types...>&& t) noexcept; // Note A
template<size_t I, class... Types>
constexpr const tuple_element_t<I, tuple<Types...>>& V&
get(const tuple<Types...>& t) noexcept; // Note B
template<size_t I, class... Types>
constexpr const tuple_element_t<I, tuple<Types...>>& V& get(const tuple<

```

~~I < sizeof...(Types)~~ ~~I~~

I t

I < sizeof...(Types) V tuple element t<I, tuple<Types...>> V

T Types X& T X&

X&& T&&

T Types X&

X& const X& T&

T

```

template<class T, class... Types>
constexpr T& get(tuple<Types...>& t) noexcept;
template<class T, class... Types>
constexpr T&& get(tuple<Types...>&& t) noexcept;
template<class T, class... Types>
constexpr const T& get(const tuple<Types...>& t) noexcept;
template<class T, class... Types>
constexpr const T&& get(const tuple<Types...>&& t) noexcept;

```

~~T~~ ~~Types...~~

t T

Types...

T

Types...

```

const tuple<int, const int, double, double> t(1, 2, 3.4, 5.6);
const int& i1 = get<int>(t); // OK. Not ambiguous. i1 == 1
const int& i2 = get<const int>(t); // OK. Not ambiguous. i2 == 2
const double& d = get<double>(t); // ERROR. ill-formed deleted

```

get

template

array

```

template<class T, size_t N>

```

```
struct tuple_size<array<T, N>> : integral_constant<size_t, N> { };
tuple_element<I, array<T, N>>::type
```

$I < N$

I

T

```
template<size_t I, class T, size_t N>
constexpr T& get(array<T, N>& a) noexcept;
template<size_t I, class T, size_t N>
constexpr T&& get(array<T, N>&& a) noexcept;
template<size_t I, class T, size_t N>
constexpr const T& get(const array<T, N>& a) noexcept;
template<size_t I, class T, size_t N>
constexpr const T&& get(const array<T, N>&& a) noexcept;
```

$I < N$

I

a

$I < N$

`<variant>`

```
// 23.7.5, value access
template<class T, class... Types>
constexpr bool holds_alternative(const variant<Types...>&) noexcept;

template<size_t I, class... Types>
constexpr variant_alternative_t<I, variant<Types...>> see_below& get(variant<Types...>&);
template<size_t I, class... Types>
constexpr variant_alternative_t<I, variant<Types...>> see_below&& get(variant<Types...>&&);
template<size_t I, class... Types>
constexpr const variant_alternative_t<I, variant<Types...>> see_below& get(const variant<Types...>&);
template<size_t I, class... Types>
constexpr const variant_alternative_t<I, variant<Types...>> see_below&& get(const variant<Types...>&&);

template<class T, class... Types>
constexpr T& get(variant<Types...>&);
template<class T, class... Types>
constexpr T&& get(variant<Types...>&&);
template<class T, class... Types>
constexpr const T& get(const variant<Types...>&);
template<class T, class... Types>
constexpr const T&& get(const variant<Types...>&&);

template<size_t I, class... Types>
constexpr add_pointer_t<variant_alternative_t<I, variant<Types...>> see_below> get_if(variant<Types...>*) noexcept;
template<size_t I, class... Types>
constexpr add_pointer_t<const variant_alternative_t<I, variant<Types...>> see_below> get_if(const variant<Types...>*) noexcept;

template<class T, class... Types>
constexpr add_pointer_t<T> get_if(variant<Types...>*) noexcept;
template<class T, class... Types>
constexpr add_pointer_t<const T> get_if(const variant<Types...>*) noexcept;
```

```
template<class T, class... Types>
constexpr bool holds_alternative(const variant<Types...>& v) noexcept;
```

T

Types...

true index()

T Types...

```
template<size_t I, class... Types>
constexpr variant_alternative_t<I, variant<Types...>>& get(variant<Types...>& v) noexcept;
template<size_t I, class... Types>
constexpr variant_alternative_t<I, variant<Types...>>& get(const variant<Types...>& v) noexcept;
template<size_t I, class... Types>
constexpr const variant_alternative_t<I, variant<Types...>>& get(const variant<Types...>& v) noexcept;
template<size_t I, class... Types>
constexpr const variant_alternative_t<I, variant<Types...>>& get(const variant<Types...>& v) noexcept;
```

~~I < sizeof...(Types)~~

v.index() I

variant

bad_variant_access

I < sizeof...(Types) V

variant alternative t<I, variant<Types...>>

V

```
template<class T, class... Types> constexpr T& get(variant<Types...>& v);
template<class T, class... Types> constexpr T&& get(variant<Types...>&& v);
template<class T, class... Types> constexpr const T& get(const variant<Types...>& v) noexcept;
template<class T, class... Types> constexpr const T&& get(const variant<Types...>&& v) noexcept;
```

~~T~~

~~Types...~~

bad_variant_access

T

Types...

```
template<size_t I, class... Types>
constexpr add_pointer_t<variant_alternative_t<I, variant<Types...>>& v> get_if(variant<Types...>& v) noexcept;
template<size_t I, class... Types>
constexpr add_pointer_t<const variant_alternative_t<I, variant<Types...>>& v> get_if(const variant<Types...>& v) noexcept;
```

~~I < sizeof...(Types)~~

v != nullptr v-

>index() == I

nullptr

I < sizeof...(Types) V

variant alternative t<I, variant<Types...>>

V

```

template<class T, class... Types>
constexpr add_pointer_t<T>
get_if(variant<Types...>* v) noexcept;
template<class T, class... Types>
constexpr add_pointer_t<const T>
get_if(const variant<Types...>* v) noexcept;

```

T Types...

return get_if<i>(v)

T Types...

T
Types...

-
- tuple_element variant_alternative

t std::tuple<UDT> get<1>(t)
UDT

get
std::get

```

template <std::size_t I, typename ...Ts,
          typename Enable = std::enable_if_t<I < sizeof...(Ts)>>
std::tuple_element_t<I, std::tuple<Ts...>>&
get(std::tuple<Ts...>& t) {
    return /*...*/;
}

// Out-of-bounds calls:
std::tuple<int> t;
std::get<1>(t); // error: no matching function for call to 'get<1>(std::tuple<int>&)'
// note: candidate template ignored:
//   std::get(array<Ts...>&)
//   could not match 'array' against 'tuple'
// note: candidate template ignored:
//   std::get(pair<Ts...>&)
//   could not match 'pair' against 'tuple'
// note: candidate template ignored:
//   std::get(variant<Ts...>&)
//   could not match 'variant' against 'tuple'
// note: candidate template ignored:
//   std::get(std::tuple<Ts...>&)
//   [with I = 1; Ts = {int}]
//   requirement 'I < sizeof...(Ts)' was not satisfied

```

std::tuple_element
std::get


```

template <std::size_t I, typename ...Ts>
    requires I < sizeof...(Ts)
std::tuple_element_t<I, std::tuple<Ts...>>&
get(std::tuple<Ts...>& t) {
    return /*...*/;
}

```

std::tuple_element_t<I,
std::tuple<Ts...>

```

template <std::size_t I, typename ...Ts,
    typename Enable = std::enable_if_t<I < sizeof...(Ts)>>
std::tuple_element_t<I, std::tuple<Ts...>>&
get(std::tuple<Ts...>& t) {
    return /*...*/;
}

template <std::size_t I, typename ...Ts>
std::enable_if_t<sizeof...(Ts) <= I>
get(std::tuple<Ts...>& t) = delete;

// Out-of-bounds calls:
std::tuple<int> t;
std::get<1>(t); // error: call to deleted function 'get'
               // note: declared here
               //   std::get(std::tuple<Ts...>&) = delete;
               //   [with I = 1; Ts = {int}]

```

```

template <std::size_t I, typename ...Ts>
decltype(auto) get(std::tuple<Ts...>& t) {
    static_assert(I < sizeof...(Ts), "tuple index is in range");
    return /*...*/;
}

// Out-of-bounds calls:
std::tuple<int> t;
std::get<1>(t); // error: static assertion failed: tuple index is in range
               // in instantiation of
               //   std::get(std::tuple<Ts...>&)
               //   [with I = 1; Ts = {int}]

```